

4.3.4. Progress

A guard that defers a provider's withdrawal until its dependents are gone delivers Theorem 70 only if it eventually releases. One relation on the fibers of a registry carries the argument

Definition 72. The *precedence relation* on the names of a registry is

$$n \prec m := p_n \cap d_m \neq \emptyset \quad (59)$$

so that n may provide a key m declares. It reads d and p alone, which by Lemma 59(5) come into existence with a fiber's entry and are never written again

Theorem 73 and Theorem 80 are established on the hypothesis that $\prec$ is acyclic, which is an assumption and not something the definition delivers, $n \prec n$ holding of a component that declares a key it provides itself. What $\prec$ orders is the two fibers' activations and not their lifetimes: $n \prec m$ says that n has to become Active before m can, whereas that a provider outlives its consumer is Theorem 70(2), a theorem about the guarded calculus.

A fiber's target view answers to the fiber that created it as well as to its providers. What a creator writes is τ_n through the primitive of Definition 52, and τ is monotone by Lemma 59(5). A creator can therefore turn its child's target view at most once over that child's whole existence

Progress is a claim that some rule applies, so it is formulated over the rules a host must offer: L-Begin, L-Leave, L-Unload, the landing rules L-Iter and L-Finish, and L-Divert. It appeals to the aborting alternative of L-Divert nowhere, which Section 4.4 puts to use.

Theorem 73. (Progress.) Assume $\prec$ acyclic, $\text{len}(e_n) \leq K$ for every n , and the set N of names the sequence ever holds (Definition 65) finite; and let every step apply a lifecycle rule. Write $S(n)$ for the number of steps acting on n and

$$V(n) := |\{t : \text{target}_n^t \neq \text{target}_n^{t+1}\}| \quad (60)$$

for the number of times its target view turns. Then

1. (No deadlock.) $\neg \text{quiet}^t$ implies that some lifecycle rule applies at γ^t
2. (Termination.) $S(n) \leq (K + 3)(V(n) + 1)$, and both $V(n)$ and $\sum_n S(n)$ are finite.

Consequently every maximal sequence of lifecycle steps ends in a quiescent state.

Proof. No deadlock. Let $\neg \text{quiet}^t$, so some fiber n satisfies neither clause of the quiet of Definition 53. Reading Table 1 against the four kinds it can then be:

- $\theta_n^t = \text{Inactive}$ with $\text{target}_n^t \neq \perp$: L-Begin applies;
- $\theta_n^t = \text{Reloading}(-, -, \omega_n)$ with $\text{target}_n^t = \omega_n$ whichever of L-Iter and L-Finish the value of $i_n^t(\gamma^t)$ selects applies;
- $\theta_n^t = \text{Reloading}(-, -, \omega_n)$ with $\text{target}_n^t \neq \omega_n$: L-Divert applies, landing that iteration rather than aborting it;
- $\theta_n^t = \text{Active}(-, \omega_n)$ with $\text{target}_n^t \neq \omega_n$: L-Leave applies.

Let no fiber be of any of these kinds, leaving some m_0 with $\theta_{m_0}^t = \text{Unloading}(-, -)$. Construct $m_0, m_1, \dots$ as follows: given m_j in Unloading, either $\neg \text{relied}_{m_j}^t$, in which case L-Unload applies to m_j and the construction stops, or there are $m_{j+1} \neq m_j$ and k_j with installed m_{j+1} and $\omega_{m_{j+1}}^t(k_j) = m_j$. In the latter case

$$k_j \in d_{m_{j+1}} \cap \text{dom}(\sigma_{m_j}^t) \subseteq d_{m_{j+1}} \cap p_{m_j}$$

the second membership being Theorem 70(3) at the episode of m_{j+1} that t lies in, so that $m_j \prec m_{j+1}$. Moreover $\text{target}_{m_{j+1}}^t \neq \omega_{m_{j+1}}^t$: an Unloading fiber is outside the union defining σ_γ , so k_j at γ^t is unprovided or provided by a fiber other than m_j . Were m_{j+1} in Active or Reloading it would then be of one of the four kinds excluded, so it is in Unloading and the construction continues. The m_j are $\prec$ -increasing, hence distinct by acyclicity, and $\text{dom}(F^t)$ is finite, so the construction stops.

Termination. Two claims bound $S(n)$.

(A) Over a maximal interval on which target_n^t is constant at ω^* , at most $K+3$ steps act on n . Reading the θ_n columns of Table 1, from $\text{Active}(-, \omega)$ with $\omega \neq \omega^*$ the fiber takes an L-Leave and an L-Unload and then, if $\omega^* \neq \perp$, an L-Begin and at most $\text{len}(e_n) \leq K$ landings; from Reloading against an $\omega \neq \omega^*$ it takes an L-Divert in place of the L-Leave, and from any other state a suffix of that sequence. No further L-Divert or L-Leave falls in the interval, the ω that the L-Begin writes being $\text{target}_n^t = \omega^*$ itself, and at $\text{Active}(-, \omega^*)$ and at Inactive with $\omega^* = \perp$ no rule applies at all.

(B) If $\text{target}_n^t \neq \text{target}_n^{t+1}$ and step t acts on m , then either $m \prec n$ or step t writes τ_n . By Definition 53 the value of target_n^t is a function of τ_n of the lifecycle states, and of the domains of the providers' tables, never of a bound value; a provider satisfies $k \in \text{dom}(\sigma_m) \cap d_n$ and hence $m \prec n$ and a table's domain changes only at a step acting on its own fiber by Lemma 59(1). Acyclicity gives $m \neq n$ in the first case, and the monotonicity of Lemma 59(5) admits the second at one t per fiber.

By (A) the interval count bounds $S(n)$ as $S(n) \leq (K+3)(V(n)+1)$, and by (B) each turn of target_n^t either consumes a step of a fiber strictly $\prec$ -below n or is the one turn τ_n affords, so $V(n) \leq 1 + \sum_{m \prec n} S(m)$. Since $\prec$ is acyclic and N is finite, the recursion

$$B(n) := (K+3) \left(2 + \sum_{m \prec n} B(m) \right)$$

is well founded and defines B with $S(n) \leq B(n)$; hence $V(n)$ is finite and $\sum_n S(n) \leq \sum_n B(n)$. By (1) a sequence that cannot be extended is quiescent $\square$

Finiteness of N is assumed rather than derived, and one condition on the components delivers it. The components a host holds are finitely many programs given before anything runs, so if no component can instantiate, however indirectly, a fiber of a component that instantiates one of its own, the instantiations form a tree of bounded depth, and $\text{len}(e_n) \leq K$ bounds its branching. What the assumption rules out is a component that instantiates itself without bound.

The target records the providing fiber rather than a boolean, and under the single-source discipline of O-Insert the two drive the same transitions, a key having one possible provider there. The view supplies the vocabulary of the results above, Theorem 70 and Theorem 71 both speaking of the resolution a fiber activated against, and it is what makes those results survive the scoped resolution of Section 3.2.3, under which one key resolves to different providers in different realms and the provisions no longer force the view. The implementation carries that scoping and holds the view in `fiber.committed` (Section 5.1.3).

4.3.5. Confluence

The results so far are about individual fibers. The property that characterizes the system as a whole is that its dynamic history leaves no trace: whatever sequence of activations and deactivations a running system has been through, the state it quiesces at is the one the same insertions and retirements would have produced had each component that ends up active been loaded once, in dependency order, and none ever unloaded. The lifecycle relation is confluent, and the normal form it converges on is the statically assembled one. This is the analogue, for dynamic composition, of the consistency with a from-scratch evaluation that change propagation establishes for incremental computation [47].

The claim is about $\rightarrow$ alone. Orchestration steps are inputs, and two sequences given different inputs land in different places for no interesting reason; what is at issue is whether the lifecycle rules, which are nondeterministic in which fiber steps next and in which exit a Reloading fiber takes, can be made to disagree. Which fiber provides a key is not among the choices, a key having one possible provider (Definition 50), so the schedule picks orders and exits and nothing else

Three lemmas are needed first. The first fixes the set of fibers that end up Active without reference to any sequence of steps, which is what makes it a function of the input rather than of the schedule

Definition 74. A fiber is *supported* at γ when it is not retired, the fiber instantiating it is supported, and every key it declares is provided by a supported fiber. The *support relation* on $\text{dom}(F_\gamma)$ is the union of the two relations those clauses read,

$$m \triangleleft n := m \prec n \vee \pi_n = m \quad (61)$$

and where it is well founded (Lemma 75) we write A for the *support set*, the fibers supported at γ :

$$n \in A := \neg \tau_n \wedge (\pi_n = \text{root} \vee \pi_n \in A) \wedge \forall k \in d_n. \exists m \in A. k \in p_m \quad (62)$$

where $\pi_n = \text{root}$ marks a fiber the orchestrator inserted and π_n otherwise the fiber whose activation instantiates n . The clauses read no field but τ, π, d, p . Both halves relate a fiber to one immediately below it, a parent rather than an ancestor and a direct provider rather than a transitive one, since that is what the clauses read; where the results below want an order they take the transitive closure, whose minimal elements, maximal elements, and linearizations are those of $\triangleleft$.

The clauses refer to A itself, so the definition is a recursion along $\triangleleft$, and it is the following that makes it one with a solution

Lemma 75. (Support is well founded.) Let $\prec$ be acyclic and let γ be reached by a sequence of steps. Then $\triangleleft$ is well founded, and A is the one solution of Definition 74, a function of τ, π, d , and p alone.

Proof. Order the names of $\text{dom}(F_\gamma)$ by the index of the O-Insert that introduced each, which Definition 58 supplies by starting the sequence at an empty registry. The parent half of $\triangleleft$ descends in that index: an O-Insert has $\pi \in \text{dom}(F_\gamma)$ as a premise, so a parent pointer names a fiber introduced earlier, and iterating it reaches the whole ancestry of a name in finitely many steps. A cycle therefore has to use $\prec$, and since $\prec$ is acyclic it has to mix the two, which needs some m to declare a key that a fiber of m 's own subtree may provide. Such a fiber is instantiated

by an activation of m or of one of m 's descendants, hence at a step after the L-Begin of m ; that L-Begin has $\gamma \models d_m$ as a premise, so a fiber providing the key is Active already before it, and clause (2) of Definition 63 leaves the key no second possible provider. The fiber that would close the cycle is therefore never introduced, and the edge is absent from $\text{dom}(F_\gamma)$. A well-founded recursion has one solution, and the clauses read the four fields alone $\square$

The last clause reads p , the keys a component may provide, whereas the target reads $\text{dom}(\sigma_\gamma)$, the keys its fibers have installed, and Definition 48 relates the two by $\text{dom}(\sigma_n) \subseteq p_n$ alone. The support set therefore over-approximates the Active fibers in general, and the condition that closes the gap is the following.

Definition 76. A component (d, p, e) is *total on its provision* when an activation of it that finishes has installed every key of p so that $\text{dom}(\sigma_n) = p_n$ at every Active fiber instantiating it

This is a condition on the components alone, mentioning no lifecycle state and no step, and independence (Lemma 66) already bounds how far it can fail: were a component to install a key only at context states another component's effects reach, its forward map would not commute with that component's, so the keys a fiber installs are fixed by its component rather than by the schedule. What totality adds is that the fixed set is all of p rather than a proper subset of it.

Lemma 77. (Support at quiescence.) Let $\prec$ be acyclic, let $\text{quiet}(\gamma)$, and let every component of γ be total on its provision (Definition 76). Then the support set is the set of Active fibers:

$$A = \{n : \theta_n = \text{Active}(-, -)\} \quad (63)$$

Proof. Write A' for the right-hand side. The quiet of Definition 53 leaves Inactive and Active as the only states and reads

$$n \in A' \iff \text{target}_n(\gamma) \neq \perp$$

By Definition 53 the right side holds exactly when $\neg \tau_n$ and every $k \in d_n$ lies in $\text{dom}(\sigma_\gamma)$, and $\text{dom}(\sigma_\gamma) = \bigcup_{m \in A'} p_m$ by Definition 76. The middle clause is the one the target no longer carries, and instantiation supplies it: a fiber with $\pi_n \neq \text{root}$ is instantiated only by an activation of π_n , and if $\pi_n \notin A'$ then π_n is not Active, so its accumulator has run and retired n by Definition 52, giving τ_n . Hence A' satisfies the clauses of Definition 74, and Lemma 75 gives them one solution, so $A = A'$. $\square$

Lemma 78. (Transposition.) Let F^t be well formed and let steps t and $t+1$ act on distinct fibers m and n .

1. If both apply an activation rule, namely L-Begin, L-Iter, or L-Finish, e_m and e_n are independent (Definition 65), and step $t+1$ is applicable at γ^t , then step t is applicable at the state step $t+1$ produces from γ^t , and the two orders reach the same γ^{t+2} .
2. If step t applies an activation rule at m , step $t+1$ an orchestration rule at n , and step t does not instantiate n , then the same holds of the two.

Proof. For (1), by Table 1 the step of m writes θ_m and, within $\Psi^t \in \mathfrak{M}(e_m)$, the tables at keys of $d_m \cup p_m$. It therefore leaves θ_n and i_n alone, and by clause (2) of Definition 42 leaves the inverse and the continuation that i_n yields alone as well, so only the premises of step $t+1$ that mention target_n remain to be checked. Its retirement half cannot fail, no activation rule writing a τ . Its resolution half cannot move either: target_n reads lifecycle states and table domains, of which Ψ^t moves $\text{dom}(\sigma_m)$ alone (Lemma 59(1)); step $t+1$ being applicable at γ^t puts every

$k \in d_n$ in $\text{dom}(\sigma^t)$, and clause (2) of Definition 63 makes the fiber providing such a k the only one that can, so $k \notin p_m$ and no domain at a key of d_n moves. The same argument in the other direction leaves step t applicable. Finally $\Psi^t \in \mathfrak{M}(e_m)$ and $\Psi^{t+1} \in \mathfrak{M}(e_n)$ commute by clause (1) of Definition 42, and the two edits write control fields of distinct fibers, so the composite is the same in either order.

For (2), the orchestration step has $\Psi^{t+1} = \text{id}_\Gamma$ by Table 1, so the two state maps commute outright, and its edit ^{$t+1$} writes τ_n or $\text{dom}(F_\gamma)$ at n alone, which the activation step neither reads nor writes: the premises of the latter read θ_m, i_m, τ_m and target_m , and an O-Insert of a fresh n moves no target, a fresh fiber providing nothing, whereas an O-Retire or O-Remove of n leaves σ_γ where it was, n being inactive in the one case and unaffected in its table in the other. So step t remains applicable. Conversely each premise of the orchestration step is either read at n , which step t does not write, or is one of the two premises of O-Insert that a smaller registry only relaxes, whence its applicability at γ^{t+1} gives its applicability at γ^t here step t not instantiating n is what keeps n present at γ^t where O-Retire and O-Remove require it. $\square$

Lemma 79. (Deletion.) Let every component be total on its provision (Definition 76), let the sequence of steps reach a quiescent γ^T , let $[b, u]$ be an episode of n that closes, let no episode of any m with $n \prec m$ close in the sequence, and let no fiber n instantiates during $[b, u]$ have an episode. Write R for the names those instantiations draw. Then deleting the steps that act on n in $[b, u]$, together with every step acting on a name of R , leaves a sequence of steps reaching a state $\simeq_K$ -equal to γ^T and $\simeq$ -equal to it outside R .

Proof. The deleted steps leave the state where they found it. Let $t_1 < \dots < t_l$ be the steps of $[b, u]$ that act on fibers other than n . Corollary 69 reads

$$\gamma^{u+1} \simeq_K (\Psi^{t_l} \circ \dots \circ \Psi^{t_1})(\gamma^b)$$

whose right side is what the surviving steps of $[b, u]$ produce on their own $\gamma^{b-1} \simeq_K \gamma^b$ and their edits writing control fields of fibers other than n that the deletion does not touch. By Table 1 the deleted steps of n edit no field but θ_n , which Lemma 59(4) restores to inactive at u and which it held at γ^{b-1} .

An invariant carries the suffix. Write γ'^t for the state the surviving steps reach at the point corresponding to t . We claim, for every $t > u$ that $\gamma^t \simeq_K \gamma'^t$, that every name of R is vestigial at γ^t and absent from γ'^t , and that the two states agree on every field of every name outside R . At $t = u + 1$ this is the paragraph above together with Definition 52, which leaves each name of R retired by the accumulator that ran at u inactive and holding an empty table, the fibers of R having no episode by hypothesis. The induction step is Lemma 62(1) applied at each name of R in turn: a step acting outside R has the same premises at the two states, reaches states again $\simeq_K$ -equal, and leaves the entries of R vestigial. A step acting on a name of R is one of the deleted ones, and Lemma 62(2) is why it has to be deleted rather than kept, an O-Retire or O-Remove of an absent name having no fiber to act on; by (1) again such a step moves no field outside R , so dropping it preserves the invariant. Hence the final states are $\simeq_K$ -equal, and equal outside R .

No surviving step loses a premise. A step acting on $m \notin R \cup \{n\}$ reads n only through $\text{target}_m(\gamma)$ or $\text{relied}_m(\gamma)$. The first depends on n when m declares a key n provides, hence $n \prec m$, and when n instantiated m , which puts $m \in R$. In the first case m 's episode does not close, by hypothesis, so it is open at γ^T , where quiet gives $\omega_m = \text{target}_m^T$ and Lemma 77 puts its values among the Active fibers, which n is not; since a key has at most one possible provider, n provided no key

of d_m at m 's L-Begin either. The second reads n only through the values of ω_n and deleting the episode can only make relied false, which relaxes the guard on L-Unload rather than blocking it. What such a step reads of a name of R is covered by the invariant. $\square$

Theorem 80. (Confluence.) Let a sequence of steps reach a quiescent γ^T , let every component be total on its provision (Definition 76), and let A be as in Definition 74. Then

1. (Canonical form.) γ^T is reached, up to the names whose entries the reduction withdraws, from γ^0 by a sequence that takes the same orchestration steps in their original order, those at a fiber the orchestrator inserted preceding every lifecycle step and each of the rest following the step that instantiated the fiber it acts on, and that takes, for an enumeration $n_1, \dots, n_k$ of A linearizing $\triangleleft$, one episode of each n_i in that order
2. (Confluence.) Any two such sequences from γ^0 taking the same orchestration steps reach states related, after a renaming as in Lemma 61, by the $\simeq$ of Definition 58 and hence by $\simeq_K$.

Proof. For (1), the episodes of the sequence are of two kinds: those that close and those still open at γ^T , which by quiet^T and Lemma 77 are one episode of each fiber of A .

Closing episodes go first, by induction on their number. At each stage pick a closing episode of a fiber n that is $\triangleleft$ -maximal among the fibers whose episodes still close; one exists by Lemma 75 and the finiteness of N . The three hypotheses of Lemma 79 are then met. No m with $n \prec m$ has a closing episode, by maximality. And no fiber n instantiates during $[b, u]$ has an episode: such a fiber is retired by the accumulator that ran at u (Definition 52) and by Lemma 59(5) stays retired, so its target view is $\perp$ and Lemma 77 puts it outside A , whence it has no episode open at γ^T ; and $\triangleleft$ relates it to n through its parent pointer, so by maximality it has no closing one either. The lemma removes the episode, together with the steps of the names it instantiated, leaving γ^T where it was up to those names. The measure drops by one, so no closing episode remains.

A fiber outside A takes no lifecycle step. It has no open episode at γ^T , by Lemma 77 and quiet^T , and no closing one now remains, so it has no episode at all and is inactive throughout; L-Begin is the only rule that applies there, and applying it would open an episode.

Orchestration steps go next. An orchestration step at a fiber the orchestrator inserted moves one place earlier past a lifecycle step of a different fiber by Lemma 78(2), which applies because a step of a fiber of A instantiates no such name: instantiations draw fresh names, whereas the name here is one an O-Insert of the original sequence introduced. With a lifecycle step of the same fiber there is nothing to exchange, an O-Insert of n already preceding every step of n and an O-Retire or O-Remove of n applying only outside A , which takes no lifecycle step. Moving each to the front in turn preserves their relative order. An orchestration step at a fiber some activation instantiated cannot go to the front, its premises requiring that fiber to be present, so it stays where the instantiation put it; it acts outside A by the paragraph above and therefore commutes with everything between it and the instantiation by the same clause of Lemma 78.

Episodes are sorted and made contiguous, by induction on $|A|$. Let n_1 be $\triangleleft$ -minimal in A . Then $d_{n_1} = \emptyset$ and $\pi_{n_1} = \text{root}$, since Definition 74 puts a provider of a key of d_n and the fiber instantiating n_1 in A while $\triangleleft$ puts both below n_1 . So target_{n_1} reads no field of another fiber and, no orchestration step remaining to write τ_{n_1} and no fiber below n_1 remaining to retire it, is constant. Every step acting on n_1 is an activation step, no episode closing, and its remaining premises read θ_{n_1} and i_{n_1} , which by Table 1 only n_1 writes; each is therefore applicable at every earlier state, and Lemma 78 moves it one place earlier without moving the endpoint. Its independence

hypothesis is met because no fiber entangled with n_1 takes a step in the region crossed — $d_{n_1} = \emptyset$ leaves n_1 no provider, and a fiber declaring a key of p_n has target $\perp$ until n_1 is Active, so its steps all follow the last step of n_1 — and Lemma 66 makes every other pair independent. The number of steps of other fibers preceding a step of n_1 drops by one at each application, so the episode of n_1 becomes an initial contiguous block. The argument repeats on $A \setminus \{n_1\}$ over the suffix that follows the block, where n_1 is Active throughout and takes no further step, so it too contributes a constant target; the providers a later n_j declares lie in earlier blocks and take no step in the suffix, so the entanglement argument above holds at every stage. The enumeration this produces linearizes $\triangleleft$ by construction.

For (2), both sequences reduce by (1) to a canonical one, and the two reductions run over the same A up to a renaming. Definition 74 reads τ, π, d and p of which the last three are written once with a fiber's entry (Lemma 59(5)), so what has to be seen is that the same names come into existence carrying the same d, p and π and that the same names are retired. Insertions the two sequences share by hypothesis. Instantiations they share as well: an activation of a fiber of A instantiates, at each of its iterations, the component the iterator names there, which the interleaved steps hold fixed — clause (2) of Definition 42 for a fiber not entangled with the activating one (Lemma 66), and Lemma 67(3) leaving an entangled fiber no $\Psi \neq \text{id}_\Gamma$ while the activation runs — so the tree of instantiations below an A -fiber is a function of that fiber's component; the names those instantiations draw are not shared, and it is here that Lemma 61 is applied, matching the two trees by a bijection. And a retirement is either an orchestration step, shared, or the O-Retire an accumulator takes, which retires exactly the names the same activation instantiated. Two enumerations linearizing $\triangleleft$ differ by transpositions of incompatible episodes; Lemma 78 leaves each endpoint unchanged up to the $\simeq$ of Definition 58, and Lemma 60 carries the steps that follow across that relation, so the two canonical sequences agree. With the termination of Theorem 73, the lifecycle relation therefore has unique normal forms. $\square$

The theorem is what licenses reasoning about a Cordis application as though it were statically assembled. An orchestrator that adds a component, removes it, replaces a provider, and undoes the replacement is guaranteed to arrive at the state it would have obtained by writing the final composition down at the outset, and a component author reasoning about which coeffects are in scope may reason about the quiescent state alone. It also delimits the guarantee: it speaks of the state, not of the emissions the system produced along the way, which is the distinction Section 6.1 draws between an acquisition, tracked inside the boundary, and an emission, which crosses it.

4.4. Extensions

We give four extensions of the calculus, each realized by the implementation of Section 5 and each leaving the results of Section 4.3 intact.

Asynchrony. The rules are synchronous: the state map Ψ^i of a step (Definition 58) is applied whole at that step, and the environment moves only between one map and the next. In an asynchronous host the iterations and the inverses yield futures, so a map in flight runs to completion whether or not it is still wanted, and the aborting alternative of L-Divert is not one such a host can offer. Such a host is *inertial*: of L-Divert it takes the landing alternative alone, and a fiber whose target view turns during an iteration deactivates after that iteration lands from Unloading, holding the inverse it produced. Inertia is therefore a restriction on which alternative of L-Divert a host may take; every result of Section 4.3 quantifies over all sequences

of steps and so covers the inertial ones, and Theorem 73 appeals to the aborting alternative nowhere, so a host bound by inertia still quiesces. An inverse in flight calls for no counterpart of inertia: the rules never decline a deactivation, Unloading being left by L-Unload alone with no premise on the target view, and a step of another fiber falling within the accumulator's application commutes with the inverses not yet applied, by the arguments Theorem 68 rests on (Definition 65, Lemma 67 Lemma 66), so an application spread over an interval reaches the state the one-step application reaches, up to $\simeq_K$. A deactivation may also chain straight back into an activation: the accumulator runs whatever the target view has become, and from Inactive an L-Begin may immediately follow, which is the mutual chaining of reload and unload in the implementation (Section 5.1.3).

Failure. The effects a component installs reach outside the context that tracks them, and a location they reach may refuse: a port already bound, a file that is not there, a peer that does not answer. Refine the iterator so that an iteration may *raise* an error in place of yielding a triple, $\Gamma \rightarrow \text{Either}(\Xi, \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathcal{J}))$ for a set Ξ of errors, the witness constraining the Right case alone, a raise having nothing to undo. A raise exits Reloading by the route of an aborting L-Divert whose premise on the target view is dropped, the iteration rather than the environment choosing the abort: the fiber routes into Unloading with the accumulator built up to the failing iteration, arrives at Inactive having installed nothing (Corollary 69), and the exit writes the error as an *outcome* on the fiber. The outcome withholds re-entry: L-Begin is read as requiring an error-free fiber, so an effect function that raised is not retried against an unchanged environment, and quiet admits a failed fiber whatever its target view; the failure likewise stays on the fiber rather than propagating to its parent, leaving siblings running. A retry is a revision: the reinserted fiber of the Configuration paragraph starts without an outcome. Preservation and recovery hold unchanged, a raise leaving by the same Unloading route every deactivation takes. Confluence excludes failed fibers, and has to: whether an iteration raises depends on the state it meets, so one schedule may fail a fiber where another completes it, and the two quiescent states then differ in that fiber's lifecycle state and, by Corollary 69, in nothing else. The FAILED state of the implementation carries this outcome (Section 5.1.3).

Isolation. Section 4.1 reads every key at one shared realm and names the relaxation a calculus carrying realms would make: provisions disjoint within a realm rather than outright, each declared key resolved against the realm of the fiber declaring it. For realms fixed at a fiber's insertion, the relaxed calculus is the present one read at a larger key set. Take the key set to be $K \times R$ each pair (k, r) carrying the value set $\mathcal{V}_k$ and the operations $\mathcal{A}_k$ of its underlying key, so that $\simeq_{(k,r)}$ is $\simeq_{k'}$ a key commutative in the sense of Definition 44 is commutative at every realm, and the coeffect at a pair inherits the witness of its underlying key (Definition 46). A fiber inserted under a realm table ρ then declares and provides pairs, a key k of its interface standing for $(k, \rho(k))$; the last premise of O-Insert, read at pairs, is disjointness within a realm two fibers providing one key in different realms provide different pairs, and the one shared realm is the diagonal (k, k) , a key outside $\text{dom}(\rho)$ resolving to its own realm (Definition 24). Keys are as atomic to the rules as names: no rule computes one, inspects its structure, or relates two of them by anything but equality, so the rules and the results of Section 4.3 apply at $K \times R$ as they stand. The reading stops at isolate itself (Definition 25), which reassigns ρ on a running context: under the pairing a reassignment moves a declaration from one pair to another, and d_n and p_n are written once with a fiber's entry (Lemma 59(5)), so a fiber whose realm turns at runtime is one whose interface has changed, a revision the Configuration paragraph carries. Interception (Definition 26) calls for no extension: metadata adjusts how a binding is used rather than what a key resolves to, is consulted when the binding is accessed, and is discarded with the context that carries it, so no premise reads it and no field of a fiber holds it.

Configuration. A component of the implementation takes a configuration, and instantiation binds the payload into the effect function the fiber runs (Section 5.1.3). The calculus therefore carries configuration inside e : one such component bound to two payloads is two components of Definition 48, differing in their effect functions. The declarative layer of Section 5.2.1 further lets the orchestrator revise a running fiber, replacing its configuration, reassigning its realms or disabling and later re-enabling it. Each revision is a composite of the rules: disabling is an O-Retire, and every other revision retires the fiber, lets the lifecycle rules deactivate it, removes the entry, children before parent as O-Remove requires, and reinserts the fiber at the same name, with the new effect function, the new realm pairs, or, at a re-enablement, the component unchanged; the name may be reissued, no stale committed view naming a removed fiber (Section 4.3.1). Dependents follow unprompted: the guard on L-Unload orders the withdrawal after the deactivations it causes, and the target-view comparison reactivates each dependent once the reinserted fiber provides the keys again. Re-enablement takes the composite rather than a rule writing τ back to $\perp$, for two reasons: Theorem 80 rests on a fiber an accumulator retires staying retired (Lemma 59(5)), and Definition 53 reads no parent pointer, so a fiber unretired after the accumulator of the fiber that instantiated it has run would activate with its creator gone. The implementation keeps the same division: re-enabling an entry instantiates a fresh fiber, the entry being the identity that survives revision and the fiber the identity of one enablement. The composite is held to its endpoint rather than to its steps: by Theorem 80 the system quiesces where a load of the revised configuration from scratch would have left it, and that endpoint is what the loader’s shorter routes answer to, a new payload handed to a component that reloads only on a material change, a realm moved without reloading its provider (Section 5.2.1).

5. Implementation and Case Study

This section presents *Cordis*, which realizes the formal models of Section 3 as a practical programming abstraction. *Cordis* is a *meta-framework* of spatiotemporal composability: unlike application frameworks that target a specific domain (e.g., web routing, ORM, UI rendering), it prescribes no concrete scenario; its sole responsibility is to supply universal dynamic composition semantics. The implementation is layered into three tiers: (1) the *core library* (Section 5.1) implements the effect and coeffect systems directly; (2) the *component loader* (Section 5.2) extends the core with configuration reconciliation and hot module replacement; and (3) application frameworks such as *Koishi* (Section 5.3) build domain-specific functionality on top of the former two tiers.

5.1. Core Library

Table 2 summarizes the correspondence between theoretical constructs and their runtime counterparts. In particular, we use the runtime names introduced below throughout this section, reserving the theoretical symbols for the formal correspondence. We also write $@name$ for a framework-internal symbol key, so the brackets in $ctx[@store]$ denote symbol-keyed access to an opaque slot on the context, rather than indexing into a string-keyed map.

Theory (Section 3, Section 4)	Implementation
Γ_∞ $\gamma \in \Gamma$	ctx, the first-class context the context tree together with everything the running system has touched
$\mathfrak{E}_\Gamma, \mathfrak{I}_\Gamma$ $\text{effect}_\Gamma(e)$ $\Sigma, \Sigma^{\text{iso}}, \Sigma^{\text{inter}}$ $\text{get}(k), \text{set}(k, v)$ $\text{isolate}(k, r)$ $\text{intercept}(k, \nu)$	Effect callback returning / yielding inverses ctx.effect(callback) ctx[@@store], ctx[@@isolate], ctx[@@intercept] ctx.get(key), ctx.set(key, value) ctx.isolate(key, realm) ctx.intercept(key, metadata)
$\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$ $\text{dom}(F_\gamma)$ $n : \mathfrak{N}$ $d : \mathfrak{D}_\Gamma$ $p : \mathfrak{P}_\Gamma$ $e : \mathfrak{I}_\Gamma^{d \cup p}$ $\pi : \mathfrak{N}$ derived realization (Definition 23) θ (Definition 49) recover, accumulator g ω (Definition 49) $\text{provider}_k(\gamma)$ $\text{target}(\gamma, n)$ Future, inertia (Section 4.4)	fiber, the instantiation of a component in $\mathfrak{E}_\Gamma$ enumerated through ctx.registry fiber.uid fiber.inject the component's provide fiber.apply fiber.parent.fiber.uid, the fiber owning the context it was instantiated on fiber.ctx, the child context the fiber runs in fiber.state, the lifecycle state, whose LOADING is Reloading and whose FAILED carries the error outcome of Section 4.4 fiber.dispose, the accumulator fiber.committed, the committed view an Impl whose provider fiber is ACTIVE fiber.target, recomputed by refresh (Algorithm 5), where $\perp$ is INACTIVE fiber.inertia, the handle of the transition in flight
O-Insert, O-Retire (Definition 52) O-Remove L-Begin, L-Iter, L-Finish L-Divert L-Leave L-Unload guard on L-Unload failure (Section 4.4)	ctx.use and the inverse of its callback (Algorithm 4) the fiber dropped from its runtime, with uid cleared execute's iteration loop (Algorithm 1) the guard failing at an iteration boundary (Algorithm 1), or reload chaining into unload refresh marking the fiber UNLOADING (Line 10) unload and its inertial chaining (Algorithm 5) unload awaiting the notified dependents (Line 25) the error recorded on the fiber, with its target set to $\perp$

Table 2 | Theory-to-implementation correspondence

The remainder of this section builds the core library from the bottom up. Section 5.1.1 realizes revertible effects, the sole primitive through which a context is mutated; Section 5.1.2 realizes reactive coefficients over it; Section 5.1.3 composes both into the component lifecycle; and Section 5.1.4 exposes the context-level operations built on them.

5.1.1. Effect Tracking

This section realizes revertible effects (Section 3.1). Every context mutation in Cordis flows through a single primitive, `ctx.effect`: coeffect provision, component instantiation, and every other context-mutating operation reduces to a `ctx.effect` call, so any operation performed through the context is automatically tracked and reverted upon component unloading. Operationally, `ctx.effect` is the realization of $\text{effect}_r^{\text{iter}}$ (Definition 18): it takes a callback of type $\mathcal{I}_r$ and lifts it to $\mathcal{I}_{\partial r}$, yielding a dispose closure that, when invoked, reverts the effect. Cordis accepts both $\mathcal{E}_r$ and $\mathcal{I}_r$ through this one operation (ad-hoc polymorphism); we take the iterator form as representative, since a plain effect function is the degenerate iterator that yields a single inverse. What the operation does not check is the witness that $\mathcal{E}_r^*$ carries: the callback supplies an inverse, and that the inverse reverts the effect it accompanies is an obligation on the component author rather than a property the runtime verifies. Theorem 68 is where the calculus appeals to it, and Section 6.1 is where the obligation is delimited. The witness of a coeffect (Definition 46) is unchecked in the same way: that the operations published at a key commute is an obligation on the component providing it, discharged by the representation choice of Section 3.4.2.

Algorithm 1 shows the construction of `ctx.effect`. We write $f \circ g$ for the disposer that runs f after g and `id` for the no-op; prepending each new inverse therefore yields LIFO recovery

Algorithm 1 Effect tracking

```

1  async function execute(callback, guard)
2    iter ← callback()
3    inverse ← id
4    while guard()
5      (value, done) ← await iter.next()
6      if value then inverse ← value ◦ inverse
7      if done then break
8    return inverse
9  function effect(ctx, callback)
10   armed ← true
11   task ← execute(callback, () ↦ armed)
12   async function dispose()
13     if not armed then return
14     armed ← false
15     recover ← await task
16     recover()
17   ctx.dispose ← dispose ◦ ctx.dispose
18   return dispose

```

The engine `execute` drives the callback as an effect iterator ($\mathcal{I}_r$, Definition 17) and folds the inverse yielded at each step into a single composite. Before each step it consults a caller-supplied guard; once the guard trips, iteration stops and only the inverses accumulated so far remain. This is the step-boundary interruption of Section 4.2.2: the `Maybe( $\mathcal{I}$ )` continuation is realized by the iterator's done flag together with guard.

`ctx.effect` is a thin wrapper over `execute` that adds two things. First, self-disposal: the guard reports the armed flag, and the returned `dispose` flips armed to false, which simultaneously

halts any in-flight iteration and makes recovery fire at most once. Firing twice would apply an inverse at a state no application of the effect produced, where nothing holds it to reverting anything. Second, parent composition: `dispose` is prepended to the enclosing context's accumulated inverse `ctx.dispose`, so a child effect's inverse is itself an effect on the parent, which is the recursive structure of $\partial^2 I$. The component level (Section 5.1.3) reuses the same `execute` with a guard that tests the stability of `fiber.target` instead of `armed`.

5.1.2. Coeffect Operations

This section realizes reactive coeffects (Section 3.2). All coeffect operations act on three symbol-keyed slots that each context carries:

- `@@store`: the value store $\sigma : (r : R) \rightarrow \mathcal{V}$ from realm symbols to typed values;
- `@@isolate`: the realm table $\rho : \text{Map}(K, R)$ from coeffect keys to realm symbols;
- `@@intercept`: the interception table $\iota : (k : K) \rightarrow \mathcal{M}_k$ assigning each key its metadata.

The first two compose into the two-layer resolution $k \rightarrow \rho(k) \rightarrow \sigma(\rho(k))$: `ctx.get(key)` (Algorithm 2) reads the realm symbol $\rho(k)$ from `@@isolate`, then the bound value $\sigma(\rho(k))$ from `@@store`. The ρ indirection lets isolation redirect a key to an independent binding, whereas `@@intercept` is consulted only when a binding is accessed, adjusting how it is used rather than what it resolves to. We realize these operations in two parts: (1) *provision and notification*, which install or withdraw bindings and propagate the change to dependents; and (2) *isolation and interception*, which reshape how a key resolves.

Provision and notification. Since `set(k, v)` has type $\mathcal{E}_\Sigma$ (Section 3.1), coeffect provision is a `ctx.effect` call and inherits its automatic tracking and recovery. Algorithm 2 implements `ctx.set(key, value)`, the concrete `set(k, v)`: the callback binds a value into the store under the realm symbol $\rho(k)$, and the returned `dispose` function removes it. Both installation and removal invoke `notify` to propagate the change to dependent components.

Algorithm 2 Coeffect operations

```

1  function get(ctx, key)
2    realm ← ctx[@@isolate][key] ▷ ρ(k)
3    return ctx[@@store][realm] ▷ σ(ρ(k))
4  function set(ctx, key, value)
5    function callback()
6      realm ← ctx[@@isolate][key] ▷ ρ(k)
7      ctx[@@store][realm] ← value ▷ σ(ρ(k) ↦ v)
8      notify(ctx, [key])
9      return function()
10       delete ctx[@@store][realm] ▷ σ \ ρ(k)
11       notify(ctx, [key])
12    return ctx.effect(callback)

```

Algorithm 3 propagates each binding change to dependents by testing, for each live fiber, whether a changed key appears in its `fiber.inject` and resolves to the same realm; if so, it calls `refresh` (Section 5.1.3) to re-evaluate that fiber against the new state, and it returns the fibers it re-evaluated so that a caller can wait for them. This is the reactive classification of Definition 22: a change that flips satisfaction activates or deactivates the fiber, and `refresh`'s idempotence

renders a neutral change harmless. The interaction of this re-evaluation with diverse control flows is developed in Section 5.1.3.

Algorithm 3 Reactive notification

```

1  function notify(ctx, keys)
2      affected ← ∅
3      for fiber in all fibers do
4          for key in keys do
5              if key ∈ fiber.inject and fiber.ctx[@@isolate][key] = ctx[@@isolate][key] then
6                  refresh(fiber)
7                  affected ← affected ∪ {fiber}
8              break
9      return affected

```

A binding counts as available to a dependent only while the fiber that installed it is ACTIVE, so refresh resolves each declared key against an active provider rather than against the store alone. This is the *provided by* relation of Definition 53, and it is what makes a withdrawal visible to dependents one step before it happens: a provider that has entered UNLOADING has stopped providing, so its dependents recompute an unsatisfied target view and begin their own teardown while its bindings are all still in place.

Isolation and interception. The two operations do structurally the same thing: each derives a child context that adjusts one inherited table for key, leaving the parent untouched, so recovery is implicit: discarding the child context suffices, with no explicit inverse to run. `ctx.isolate(key, realm)` overrides the realm mapping ρ with realm , or a freshly generated symbol by default (realizing `isolate`, Definition 25), so two contexts that assign different symbols to the same key resolve to independent bindings. `ctx.intercept(key, metadata)` merges `metadata` into the interception table ι (realizing `intercept`, Definition 27): following that definition, the new `metadata` is combined with whatever the context already carries for `key` and takes priority over it.

5.1.3. Component Lifecycle

A component is instantiated as a *fiber* by `ctx.use`. This section gives the fiber (introduced in Section 5.1) operational meaning as the inertial state machine of Section 4.4. Two fields drive the algorithm below: `fiber.parent`, the parent context of `fiber.ctx` that forms the component hierarchy (the recursive structure of Γ_∞ , Section 3.3.1), and `fiber.inertia`, a handle to the in-flight asynchronous transition (or null if idle).

Algorithm 4 shows component instantiation. A *component* pairs a coeffect specification `component.inject` (d) with an effect function `component.apply`; instantiation binds the component's config into `fiber.apply` (Line 9), the config-applied effect function (e) that the lifecycle then runs. The callback function (Line 2) is the effect tracked in the *parent* fiber: when executed, it initiates the child's lifecycle by calling `refresh` (Algorithm 5); when reverted, it forces the child's target to $\perp$ and triggers `unload`. This is the instantiation primitive of Definition 52, with `callback` as its O-Insert and the closure `callback` returns as its O-Retire: an instantiation is an ordinary tracked effect of the parent, so unloading a parent cascades to its children.